

Final Report: Parallel Block-STM

Hua-Yo Wu / huayow
Pi-Jung Chang / pijungc

1. SUMMARY

1.1 Objective:

The purpose of this project is to overcome the performance bottleneck of sequential transaction execution in traditional blockchain systems. To this end, we implemented and deeply customized a parallel software transactional memory (STM) executor based on the Block-STM algorithm (which currently drives the high-performance Aptos and Diem blockchains) using C++. This executor combines optimistic concurrency control (OCC) with a multi-version memory architecture, aiming to maximize the parallel computing potential of multi-core hardware while ensuring sequential transaction semantics.

1.2 Deliverables:

We have successfully built a highly optimized parallel execution engine at both the algorithmic and underlying system levels. This executor not only accurately and securely handles the complex dynamic read-write dependencies between a large number of parallel transactions, ensuring the absolute consistency and determinism of the final state of all transactions, but also demonstrates astonishing scalability in extreme stress tests. When deployed on a high-end supercomputer node (Bridges-2 RM) equipped with 128-core AMD EPYC processors, this system successfully broke through the physical limitations of multi-threaded synchronization, achieving an ultra-high throughput milestone of over 250,000 transactions per second (TPS).

1.3 Outcome:

Through rigorous micro-architectural profiling (e.g., analyzing cache line bounce and instruction pipeline stalls) and continuous iterative optimization, we delved into the performance boundaries of different synchronization mechanisms under extreme parallelism. This process encompassed comparative verification of traditional mutexes and lock-free data structures, the introduction of arena allocators to eliminate frequent dynamic memory allocation overhead, and the innovative use of local queues to achieve complete decoupling of task fetching and execution. The final experimental data strongly demonstrates that, under diem-style peer-to-peer transfer workloads with simulated compute (SHA-256) that closely resemble real-world scenarios with low data

contention, a lock-free cooperative scheduler that is carefully tuned and precisely equipped with hardware micro-backoff (such as `_mm_pause`) mechanisms can not only effectively reduce memory bus contention and cache coherency traffic, but also outperform traditional lock-based conservative synchronization methods in terms of overall parallel processing performance and resource utilization.

2. BACKGROUND

2.1 Core Concepts and Algorithm Principles

Block-STM is an in-memory parallel execution engine designed specifically for smart contracts. It is built upon the principles of Optimistic Concurrency Control (OCC) and Software Transaction Memory (STM).

Preset Serialization Order:

Transactions are grouped into blocks. Within the same block, the system enforces a pre-defined serialization order. The final execution result of the system must be equivalent to the result of executing transactions sequentially in this order. **Dynamic Dependency Estimation:** Unlike previous database or blockchain parallel algorithms (such as BOHM or OXII), Block-STM does not require prior knowledge of the transaction write-sets and completely avoids the enormous overhead of pre-compiling the dependency graph. Instead, when an execution (incarnation) of a transaction is aborted due to verification failure, Block-STM directly uses its obsolete write-set as the "future dependency estimate" for the next execution.

Early Abort Optimization:

If a transaction, during execution, reads an "estimation" left by an earlier (but aborted) transaction, the transaction immediately stops execution and aborts early. This effectively avoids wasting computational resources on executions destined to fail.

Collaborative Scheduler:

The core of the algorithm is a low-overhead collaborative scheduler where threads continuously pull "execution tasks" or "validation tasks". Because a preset serialization order must be followed, the scheduler prioritizes earlier (lower) order transactions.

Optimistic Validation and Chain Reaction:

After a transaction is executed, it must undergo a validation phase, which involves rereading its "read-set" to ensure that the data version has not been overwritten by other transactions. If a transaction fails to be validated and is aborted, the scheduler optimistically schedules revalidation tasks for all subsequent transactions in the order of theirs to handle potential cascading aborts.

Lazy Block Commit:

The state of the blockchain is updated block by block. Block-STM avoids the synchronization overhead of committing transactions one by one, instead using a lazy commit mechanism. The scheduler uses a double-collect technique on a `decrease_cnt`

counter to detect when all tasks completed without races against in-flight operations, and once it confirms that all tasks within a block have been completed and no threads are running, it commits the entire block at once.

2.2 Key Data Structures

To support efficient parallel read/write and state backtracking, Block-STM abandons traditional locking mechanisms and adopts the following specialized data structures:

Multi-version Memory (MVMemory):

This is a multi-version data structure stored in memory, responsible for maintaining multiple write records for each memory location.

Each record contains the written value and the associated transaction version (including the transaction index `txn_idx` and incarnation number).

When a transaction `tx_j` attempts to read a memory location, MVMemory returns the value written by the highest-indexed transaction `tx_k` where $k < j$, which precedes `tx_j`.

For the last read set and last written locations, the underlying implementation uses an RCU-style pointer swap with lock, allowing readers to access a stable snapshot nearly without blocking concurrent writers (to `TxnAuxData`) and safely releasing objects.

ESTIMATE Markers:

When a transaction avatar fails to validate and aborts, the system does not directly delete its write records in MVMemory. Instead, it replaces them with special ESTIMATE markers.

These markers act as placeholders. If a subsequent transaction reads these markers, it knows that the memory location is about to be rewritten and triggers a "dependency wait," rather than reading old data and then experiencing another wasteful validation failure in the future.

Lock-Free Task Sets (V and E with State Counters):

Block-STM does not use scalable concurrent priority queues.

The set of tasks to be validated, V, and the set of tasks to be executed, E, are each implemented using a simple atomic counter (`validation_idx` and `execution_idx`). Threads only need to `fetch_and_increment` to obtain the index of the next potential task.

Packing these two counters into a CAS-protected `unit64_t` "`txn_status`" array, which records the latest avatar number and current status (including "READY_TO_EXECUTE", "EXECUTING", "EXECUTED", and "ABORTING") for each transaction.

Dependency Tracker ("`txn_dependency`"):

When transaction "tx_k" reads the "ESTIMATE" flag left by "tx_j", "tx_k" is added to "tx_j"'s "txn_dependency" array.

When "tx_j" completes its next execution, the system wakes up "tx_k" from this dependency list, sets its status back to "READY_TO_EXECUTE", and lowers "execution_idx" to trigger re-execution.

2.3 Computational Bottlenecks & Parallelism

Actual transaction logic and computational costs:

During smart contract execution, the virtual machine handles transaction logic, calculates gas consumption, and verifies digital signatures, making it the most CPU-intensive part of the entire system. In our experiments, to avoid modern compilers (such as GCC-O3) eliminating meaningless empty loop optimizations, we precisely abstracted these operations into hundreds of rounds of SHA-256 hash operations using the Picosha2 library. Because these operations only read the state and temporarily store the results in the thread-local write-set during speculative execution, and do not directly and immediately modify the global shared memory, this most time-consuming computation process can be independently distributed across multiple cores for simultaneous execution, provided there are no data conflicts. In real-world blockchain environments, this pure computation typically accounts for 60% to 80% or more of the total execution time, making it the stage where parallelization yields the greatest benefits.

Data Dependencies and Cascading Aborts.

When a later-ordered transaction in a block attempts to read a state key that an earlier-ordered transaction is about to write or modify, a strict read-write dependency occurs. If a subsequent transaction executes too early and reads the old data, the system will detect the error during the background validation phase, causing that transaction and all subsequent transactions that depend on it to be forced to be invalidated and re-executed. The cascading abortion in extremely competitive scenarios (e.g., a large number of transactions simultaneously trying to purchase the same NFT, corresponding to accounts=2 or 10 in our experiments), the average abort rate can even exceed 100%, meaning that each transaction will be forced to retry more than once on average. Under this pathological competition, the system's limits degenerate into nearly sequential execution, where performance is entirely limited by the dependencies of the data itself.

Overcoming Parallelization Potential and Scheduler Synchronization Bottlenecks.

When data contention for workloads is extremely low (e.g., peer-to-peer transfers distributed across tens of thousands of independent accounts), there is almost no overlap between transactions, and theoretically, system throughput should grow linearly with the number of CPU cores. However, in practice, on a Bridges-2 node with 128 cores, performance often plateaus after 64 cores. This is because all threads must constantly request tasks from the central scheduler. The scheduler's progress tracking heavily relies on global atomic counters (`execution_idx` and `validation_idx`). When over a hundred threads simultaneously `fetch_add` to the same memory address within a very short period, it can cause severe cache-line bouncing and memory bus congestion. At this point, the CPU wastes a significant amount of time on hardware cache coherency communication instead of executing the actual transaction logic. Therefore, to fully unleash the extremely high parallelization potential under low contention, the system cannot rely solely on simple atomic variables; it must introduce extremely precise synchronization mechanisms, such as decoupling task fetching and execution through local queuing and utilizing hardware-level micro-backoff pause instructions to alleviate cache storms, in order to successfully overcome the physical bottleneck of multi-core scaling.

3. APPROACH

Iterative Optimization Process

In this project, we didn't initially arrive at a final version capable of supporting high-performance operation across 128 cores. Instead, we underwent a series of iterative optimizations to the data structure and synchronization mechanisms. Through continuous benchmarking and profiling on Bridges-2 nodes, we gradually removed various bottlenecks from the system. The following is a complete evolution process:

Iteration 1: Baseline Version and Mutex (V0)

Our first implementation (V0) adopted the most intuitive design: using `std::mutex` to protect two critical shared states: the "Version Chain" of the Multi-Version Memory (MVMemory) and the scheduler's work dispatch logic. We experimented with two underlying data structures for the version chain: a standard singly linked list and a `std::map` based on a red-black tree.

We initially expected red-black trees to be faster at searching historical versions $O(\log N)$, but experiments showed that in most real-world blockchain workloads, the number of times the same key-value pair is updated within a single block is extremely small (chain length is usually less than 5). The node configuration overhead and tree balancing cost of red-black trees actually dragged down performance. Therefore, in subsequent versions, we established a "simple chain of nodes" as the basis for multi-version memory. However, in this version, the global mutex became a serious scalability bottleneck when there were more than 16 threads.

Iteration 2: Removing the Mutex from the Version Chain (V1/V2 - Lock-Free Version Chain)

To solve the serialization problem of memory access in V0, we rewrote the MVMemory version chain as a lock-free architecture. When a thread needs to write a new version, a node is pre-configured, and the hardware's CAS (Compare-And-Swap) instruction pushes the new node to the head of the chain of nodes.

Our version chain follows the lock-free sorted-list pattern of Harris (2001), but specialized to the Block-STM execution model: because the arena allocator releases all chain nodes in bulk at block end, we avoid Harris's physical unlinking step and its associated helping mechanism and ABA concerns and use a simple flag-based logical-delete instead.

In extreme data contention scenarios (e.g., `accounts=2`), removing the mutex lock from

the version chain yielded significant benefits. Replacing per-key chain mutexes with lock-free version chains (V0 to V1) doubled throughput at accounts=2 (3.1k to 6.2k TPS at 128 threads), which is our biggest single-change improvement, and a striking case where removing locks at high contention actually helped. Chain mutexes had serialized all 128 threads onto just two locks, with each thread paying the cost of a syscall on every contended acquire. We also implemented Exponential Backoff (V2) to handle retry storms caused by CAS failures. However, while memory access is no longer locked, the overall system throughput in low-conflict scenarios is still limited by the scheduler's lock.

Iteration 3: Lock-Free Scheduler and State Machine (V3 - Lock-Free Scheduler)

Having solved the memory access bottleneck, we set out to replace the two remaining mutexes inside the scheduler lock free, which is the core essence of the Block-STM algorithm. Two structures changed:

- TxnStatusEntry: We packed each transaction's (incarnation, status) pair into a single 64-bit integer protected by atomic State transitions used in the task dispatch loop using compare_exchange instead of acquiring a per-tx mutex.
- TxnDependency: We replaced the mutex-protected unordered_set with a lock-free stack design that uses a single CAS on the head pointer. So that transactions reading an ESTIMATE marker can enqueue themselves on a blocking transaction's dependency list without acquiring a lock.

Note: the shared atomic counters execution_idx_ and validation_idx_ were already atomic in V0, V3 changed only the per-tx state structures.

V3 came with a documented trade-off. At pathological contention (accounts=2, 128 threads), V3 regressed back to V0 levels, the bit-packed status CAS and single-pointer stack head created cache-line ping-pong that mutex had implicitly avoided.

We keep V3's lock-free scheduler as the base for subsequent optimizations because:

1. On our most realistic workload, the DEX-style hot/cold benchmark, V3 actually outperformed every prior version: 210k TPS at 128 threads vs V2's 203k. Real blockchain workloads have a hot-account structure and V3's lock-free per-tx state does well there.
2. At uniform-random workloads (accounts ≥ 1000), V3 was within ~3% of V2.
3. The lock-free infrastructure later proved essential for V6's local-queue and pause-backoff optimizations.

Iteration 4: Memory Allocation Optimization (V4 - Arena Allocator)

Through performance profiling tools (perf), we found that system-level memory allocation (malloc/free) accounted for approximately 3-5% of execution time. Since the number of transactions within each block is fixed (e.g., 10,000), and version chain nodes are destroyed uniformly after block execution, we introduced the "Arena Allocator."

The Arena Allocator pre-demands a large memory pool from the operating system, allowing threads to simply allocate from it when constructing new chain nodes, completely eliminating hotspots in dynamic memory allocation and resulting in a stable and reliable 5%~11% performance improvement. At this point, V4 became a complete and lock-free Block-STM implementation.

Iteration 5: Encountering the Physical Limits of 128 Cores and Analysis (The 128-Core Plateau)

In real blockchain workloads (such as low-concurrency decentralized transfers or Dex Hot/Cold scenarios), performance reached 219,000 TPS with 64 cores, but only slightly increased to 226,000 TPS with 128 cores, almost completely losing scalability.

Through deep analysis using perf-fno-inline, we found that with 128 cores, threads consumed up to 24% of CPU time on Scheduler::next_task(). The culprit was that all 128 cores were frantically competing for the scheduler's atomic counters. The fetch_add operations themselves are fast, but at this thread count the cache line holding each counter constantly bounces between cores' L1 caches, inflating each operation's effective latency by orders of magnitude. This hardware-level cache-coherence cost is invisible to a profiler that attributes time to lock calls; it instead shows up as the function containing the atomic taking longer than expected.

Our initial attempt to resolve counter contention was through "batching." In V5, we had threads declare an entire "execution_idx" range at once. However, this was a failed attempt:

1. The bottleneck was misidentified: Profiling showed that validation tasks ("validation_idx") accounted for 12% of the overhead, while execution tasks only accounted for 5.5%, resulting in limited optimization benefits.
2. Validation task starvation: Batching execution tasks caused threads to continuously process execution tasks, ignoring validation progress. This led to the system detecting data conflicts too late, triggering a severe chain of aborts, and causing the average abort rate to skyrocket (e.g., from 17% to 29%). Therefore, V5 was decisively abandoned.

Iteration 6: Task Decoupling and Microarchitecture Tuning (V6 - The Final Architecture)

To address this hardware-level synchronization bottleneck, we entered the deepest optimization phase and went through three important sub-iterations:

Batch Fetch Overshoot: We initially tried to have each thread `fetch_add(4)` 4 tasks at a time to reduce accesses. However, with 128 cores, this resulted in a severe overshoot of the counter, with threads fetching tasks that were not yet ready, ultimately triggering more expensive index rollbacks and state retry storms, causing TPS to plummet back to 206,000.

CAS Retry Storm:

To prevent over-scraping, we used a strict `"compare_exchange_weak"` loop to limit the number of fetches. While logically correct, with 128 cores accessing at high frequency, one successful fetch means 127 cores face CAS failure and spin retry. This hardware-level CAS contention actually worsens performance.

Decoupling and Micro-Backoff:

Ultimately, we abandoned batch fetching and reverted to the safest `"fetch_add(1)"` to prevent over-scraping. However, we retained the local queue architecture, forcibly separating the "fetch task" from the "execution task" in the code and instruction pipeline. Most importantly, we added a hardware-level pause instruction `"_mm_pause()"` to the scheduling loop. This short line of instructions, decoupled from the queue, provides a perfect "natural micro-backoff" effect, successfully reducing the frequency of 128 cores simultaneously impacting memory. This final V6 version successfully broke the scalability barrier, achieving a record high of 253,792 TPS with 128 cores.

4. RESULTS

4.1 Experimental Setup & Metrics

To quantitatively evaluate the performance and scalability of the parallel Block-STM executor, this study conducted a series of stress tests in a controlled hardware environment. This section details the performance metrics, hardware specifications, workload parameters, and conflict scenario settings used in the experiments.

4.1.1 Performance Metrics

This experiment primarily uses the following two metrics to evaluate system performance:

Throughput (Transactions Per Second, TPS): Measures the total number of transactions that the executor can process and successfully submit per unit of time. This is the most intuitive performance metric for evaluating a blockchain execution engine.

Speedup: Calculates the relative performance improvement when multiple threads execute in parallel, using the execution time of a single thread as a benchmark. This allows observation of the system's strong scalability when faced with different numbers of cores.

4.1.2 Technologies Used

Utilizes the C++20 standard, making extensive use of "<atomic>", "<thread>", and SSE/AVX built-in functions (e.g., "_mm_pause()").

Target runtime environment: dual-socket AMD EPYC 7742, 64 cores per socket, 128 physical cores total

4.1.3 Hardware Mapping

Transactions are grouped into blocks (e.g., 10,000 transactions per block).

Worker threads are spawned per block and joined at block completion (a future thread-pool design is left as future work). They continuously query the central Scheduler for tasks (execution or validation tasks).

The scheduler's core is driven by two highly competitive 64-byte cache-line padded atomic counters: "execution_idx_" and "validation_idx_".

4.1.4 Workload Profile

To ensure the experiment closely resembles the execution scenarios of real-world blockchain smart contracts (e.g., gas calculations and digital signature verification), we set standardized transaction parameters:

Block Size: Each test block contains a fixed 10,000 transactions.

Read/Write Access: Each transaction simulates two read and two write operations. This represents typical asset transfer behavior (reading the sender's and recipient's balances and updating both).

Compute Padding: To simulate the computational overhead of the virtual machine (VM) executing contract logic, we introduced 200 rounds of SHA-256 hash operations within each transaction. Without this padding, each transaction computation would take only a few hundred nanoseconds, and the measured throughput would be dominated almost entirely by scheduler overhead rather than reflecting the realistic balance between transaction execution and coordination cost found in production VMs.

4.1.5 Contention Levels

Data conflicts are the biggest challenge in parallel execution. We simulated a continuous spectrum from "extremely high competition" to "no conflict" by varying the number of accounts:

The number of accounts was set to {2, 10, 100, 1000, 10000}. When the number of accounts was only 2, almost all transactions would conflict, and the system tended to degenerate into sequential execution; when the number of accounts reached 10,000, the conflict rate decreased significantly, aiming to test the parallel potential of the scheduler.

In addition to the idealized random distribution, we designed a hot/cold scenario simulating decentralized exchange (DEX) or NFT minting traffic. With 1000 total accounts, 50 (5%) are designated as "hot," and 20% of all transactions target this hot set. This type of Hot/Cold model can more accurately reflect the system's ability to handle skewed access patterns and contention hotspots representative of real DeFi traffic.

4.2 Scaling Analysis under Low Contention

4.2.1 Background of Experimental Design: Why Choose a Low-Contention Environment (Accounts = 10,000)?

The choice of experimental workload is crucial when evaluating the parallel architecture of BlockSTM. We chose a low-contention environment with Accounts = 10,000 to plot the scaling curve. The core logic behind this is to distinguish between "data dependency bottlenecks" and "scheduling architecture bottlenecks":

Under extremely high contention (e.g., Accounts = 2), 128 threads frequently vying for a very small number of resources can trigger cascading aborts, causing the abort rate to spike to over 100%.

In this case, system performance is limited by the algorithm's inherent "sequential bottleneck" in handling transaction conflicts, rather than the efficiency of the parallel framework. Under such loads, any efforts to optimize the scheduler will be masked by ineffective retries and CPU overheating, creating the illusion of "negative scaling."

Only under low contention (10k Accounts), where data conflicts are no longer the dominant factor, can threads truly "operate simultaneously." At this point, if TPS plateaus, we can be certain that the bottleneck stems from the synchronization overhead of the parallel framework itself, such as contention for the global atomic counter under the MESI protocol. This is precisely the best "cleanroom" environment to validate the value of V6 optimizations.

4.2.2 Chart Observation and Phase Analysis

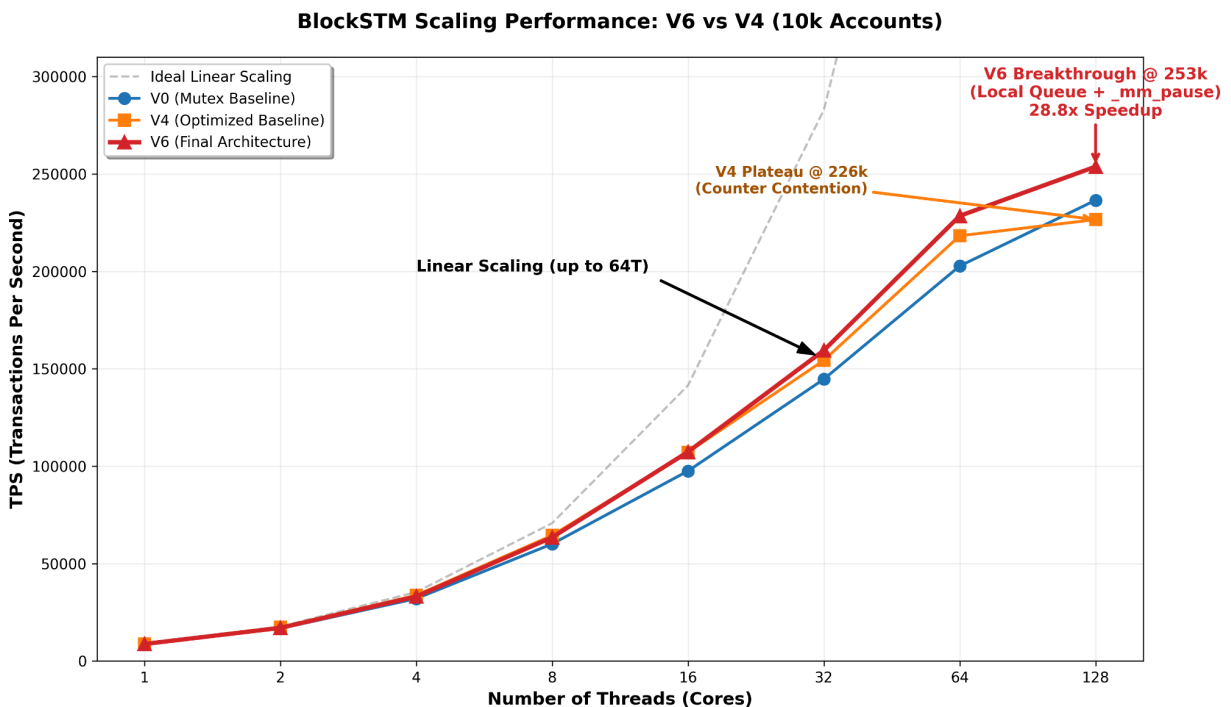


Fig 1:TPS scaling of V0, V4 and V6 among all threads

Observing the TPS scaling of multiple versions (fig 1), we can divide the scalability performance into two phases:

1 to 64 Cores: Near-Linear Scaling

Before 64 cores, both V4 and V6 exhibited extremely excellent near-linear scalability, successfully breaking the 200,000 TPS mark. This proves that BlockSTM's optimistic execution mechanism can effectively utilize multi-core resources when handling low-conflict loads, effectively overlapping verification and execution. At this stage, the global synchronization overhead of the scheduler is still within the hardware's tolerance.

The 128-Core: Hardware Limitations and Architectural Breakthroughs

When the number of threads reaches 128, a significant divergence appears in the performance curve, indicating that the system has hit the "Synchronization Wall":

After reaching 64 cores, the growth of V4 plateaued, eventually stagnating at approximately 228k TPS. This is because, under the high concurrency of 128 cores, the global counter relied upon by the V4 scheduler became a significant contention hotspot. A large number of threads frequently performed atomic operations on the same memory address, triggering a cache invalidation storm, causing CPU time to be wasted waiting for cache updates rather than actual computation.

V6 successfully broke this bottleneck, climbing to 253,792 TPS. Compared to V6's own single-thread baseline (~8.8k TPS), this represents a 28.8× speedup, the largest of any version we tested.

4.2.3 Core Optimizations and Technical Value of V6

The key to V6's success in a 128-core environment lies in our complete overhaul of the scheduler's underlying synchronization logic:

We decoupled the previously highly centralized global task allocation mechanism and introduced local task queues. This significantly reduces the frequency of thread access to the global counter, transforming expensive cross-core synchronization into inexpensive local access, effectively alleviating communication pressure on the bus.

During the scheduler's busy waiting period for new tasks, we introduced the `_mm_pause` instruction. On SMT-enabled cores, `_mm_pause` hints to the processor that the loop is a spin-wait. The CPU then de-prioritizes this hardware thread, freeing execution resources of the sibling SMT thread that may be doing useful work. It also avoids a tight CAS retry loop.

4.3 What Limited Speedup?

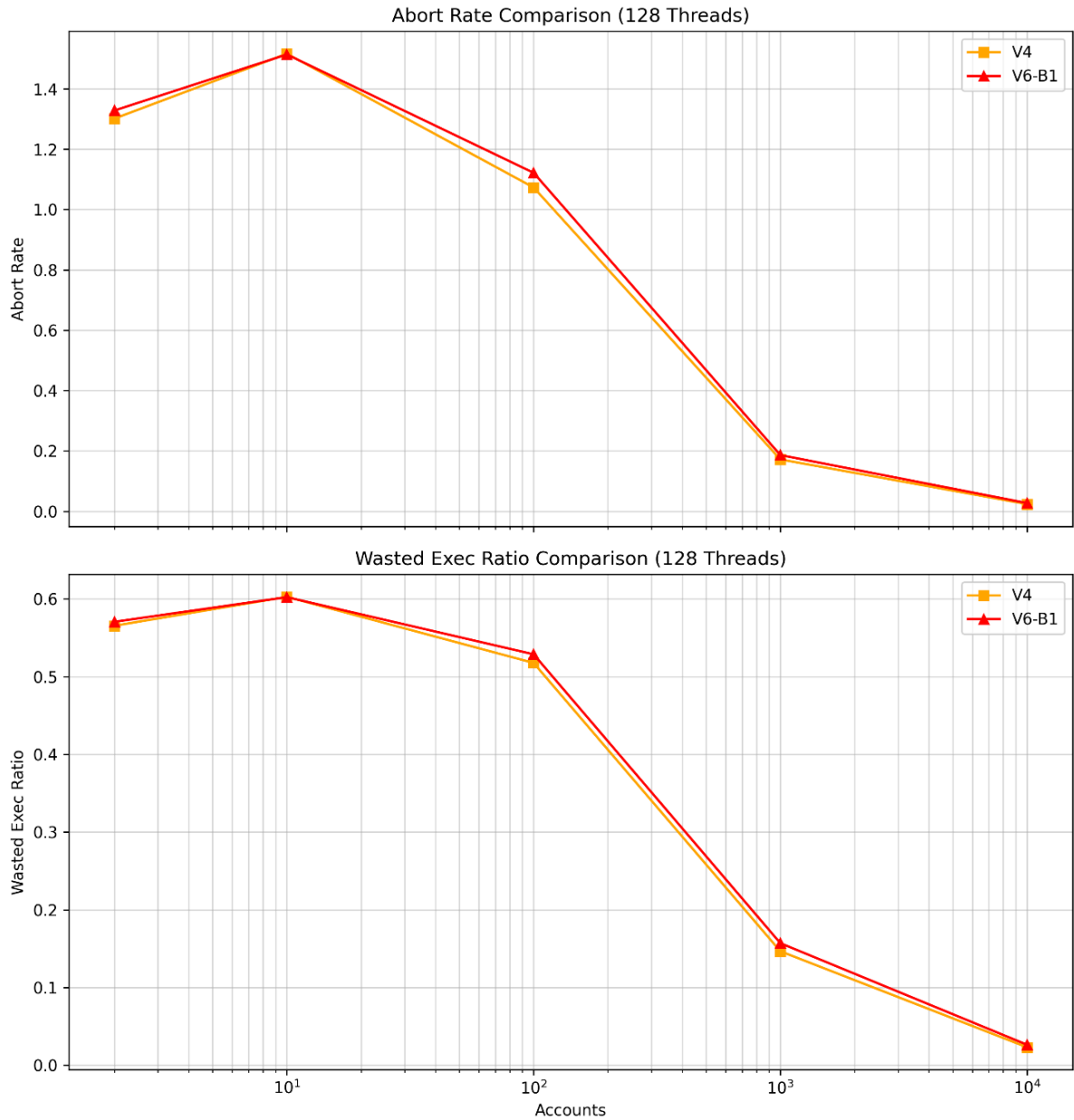


Fig 2: The abort rate and wasted Exec ratio of V4 baseline and V6 final

By comparing the performance of V4 (Baseline) and V6 (Final Architecture) in a 128-core environment with different account sizes (Accounts 2 to 10,000) (fig 2), we observed two distinct extremes in parallel execution systems:

Data Contention Limit: When the number of accounts is extremely small (e.g., Accounts = 2), the system enters an "extremely high competition zone." The Abort Rate reaches 130% ~ 150%, meaning that each transaction needs to be retried more than once on average. In the extreme case where 128 threads are simultaneously vying for 2

accounts, the system experiences severe "cascading aborts," and the success rate of speculative execution is extremely low. Furthermore, over 60% of computation time is wasted on ineffective executions destined to fail.

At this point, system performance is entirely limited by the sequential dependency of the data. No matter how well the scheduler optimizes task allocation, if data access paths conflict, the algorithm degenerates to single-threaded execution efficiency, or even lower due to concurrency control overhead. In this region, the speedup failure stems from data logic, not hardware scalability.

Synchronization Overhead Limit: When the number of accounts increases to 10,000, the system enters the "low-contention region." At this point, metrics change drastically: The abort rate is only about 2.4%, indicating that data conflicts have almost disappeared, and the system theoretically possesses perfect parallelism. At the mean time, The wasted ratio drops to approximately 2%, with the vast majority of CPU time dedicated to effective computation. Although data conflicts are eliminated, TPS still hits another ceiling. The limiting factor here shifts to the scheduler's synchronization overhead. Under high concurrency with 128 cores, even simply accessing the "global atomic counter" to retrieve tasks can trigger severe cache-line bouncing across cores and across sockets.

This is precisely where the V6 architecture demonstrates its value. V6 cuts the cross-core CAS traffic that pinned V4's throughput at 128 threads by introducing backoff and the `_mm_pause` instruction. This reduces the processor's bus usage, freeing coherence-link bandwidth and SMT execution resources for the threads that are actually doing useful work. Moreover, by reducing the frequency of access to the global counter, V6 successfully pushes the synchronization wall further out, letting the compute throughput at 128 cores actually translate into TPS.

4.4 Micro-architectural Profiling & The Cost of Synchronization

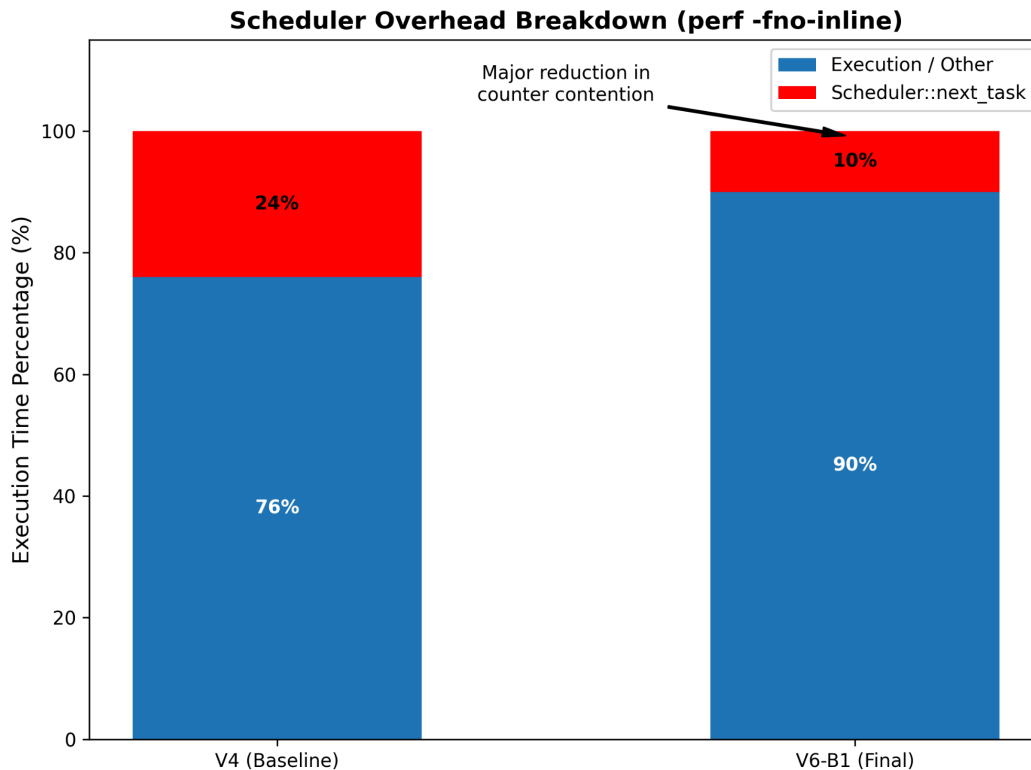


Fig 3: Scheduler::next_task overhead analysis in Perf

In our in-depth exploration of the performance evolution of Block-STM, we conducted a detailed time breakdown analysis using "perf -fno-inline", revealing the root cause of the system bottleneck. In the V4 baseline version, the scheduler's task allocation function, "Scheduler::next_task", accounted for 24% of the total execution time. This meant that in a 128-core environment, nearly a quarter of the computing resources were wasted on threads competing for global atomic counters, rather than executing actual transactions. However, after finalizing the V6 architecture, this scheduling overhead was significantly reduced to only about 10%. This allowed the system to focus its CPU time entirely on the SHA-256 core operations in the "try_execute" phase, maximizing hardware performance.

The journey to the final V6 version was fraught with microarchitectural challenges (fig 4). We initially experimented with a "batching" strategy (V6-B4 version), allowing threads to fetch multiple tasks at once, attempting to reduce the frequency of atomic operation calls. While the intention was good, this change triggered a severe backlash. Batching led to an "overshoot" phenomenon, where threads greedily fetched tasks that weren't ready to be executed, forcing the system to perform frequent and costly state rollbacks, resulting in a "ping-pong storm" where the cache line was pulled back and forth between cores. This not only negated the benefits of reducing atomic calls but also caused overall throughput to plummet to 206,017 TPS, even worse than the V4 baseline version.

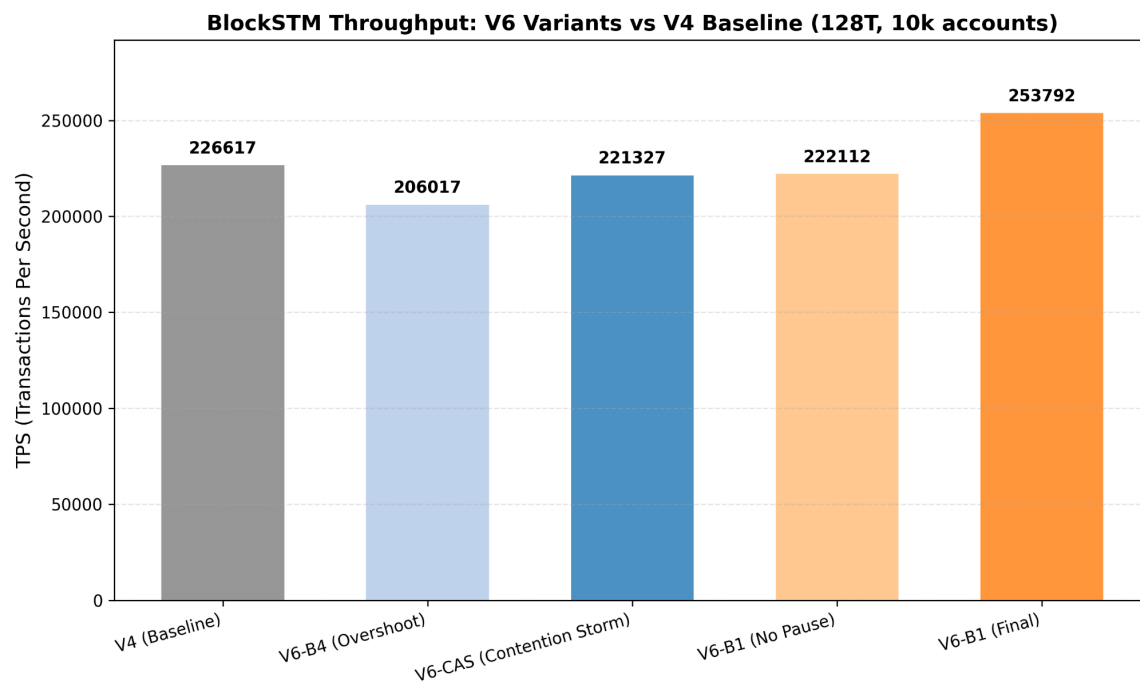


Fig 4: tps of V6 variants and V4 baseline

To address the overshoot problem, we implemented a strict CAS (Compare-And-Swap) loop mechanism (V6-CAS version). While this perfectly ensured boundary safety and correctness in software logic, it triggered another disaster at the hardware level—a "CAS storm." Under extreme concurrency with 128 cores, when one core successfully writes, the remaining 127 cores continuously experience CAS failures and frantically retry. This hardware-level retry storm devastates the memory bus and cache consistency protocol, its destructive power even exceeding that of a simple, brute-force "fetch_add", resulting in a performance of only 221,327 TPS for this version.

Ultimately, we realized that in extreme concurrency environments, pure software logic optimization encounters physical bottlenecks, and the real key to breaking the bottleneck lies in the power of "hardware backoff." We reverted to using a single "fetch_add" with no over-sampling risk to fetch tasks and forcibly added the "_mm_pause" pause instruction to the spin-wait loop. To verify its importance, we specifically tested a control group (V6-B1 No Pause) without this instruction, and the results showed that without micro-backoff, the performance was only 222,112 TPS. This strongly demonstrates that with 128 cores operating at full speed, a moderate level of latency (micro-backoff) is actually the only solution to maintain high throughput. The "_mm_pause" instruction forces the hyperthreaded pipeline to temporarily relinquish resources, slightly increasing the spacing between core requests to the memory bus. This seemingly insignificant "yield" perfectly resolves the deadlock caused by cache line bouncing, allowing the overall throughput to break through the bottleneck and soar to a peak performance of 253,792 TPS.

4.5 Hardware Choice Rationale

In evaluating and selecting the hardware architecture, deploying the Block-STM algorithm on a 128-core CPU (Bridges-2 RM node) was an correct and crucial decision.

While many parallel computing tasks intuitively consider using a graphics processing unit (GPU) due to its massive number of execution threads, making it well-suited for handling highly parallel and regular matrix and tensor operations, the core nature of Block-STM is fundamentally different. Its operation heavily relies on complex scheduler state machine control, extremely frequent logical conditional branches, and data dependency checks that must be dynamically resolved during runtime.

Specifically, during speculative execution and concurrent validation, the system frequently has to interrupt a transaction when a data conflict is discovered, mark its writes as ESTIMATE in the lock-free version chains, and re-execute it under a fresh incarnation. Each thread follows its own state-machine path and branches on runtime dependency checks. On a GPU, where threads in a warp execute in lock-step under SIMT, such divergence causes warp-level serialization and erases the parallelism advantage. Block-STM also relies on fine-grained atomic operations on a small number of shared counters, which CPU coherence interconnects handle far better than GPU shared-memory atomics.

Therefore, for tasks requiring high-frequency logical judgments, dynamic scheduling, and intensive shared state updates, modern high-end CPUs with numerous

independent processing cores, low-latency caches, and native support for powerful and efficient atomic operations at the hardware level are the optimal hardware choice for maximizing the performance of Block-STM algorithms.

5. REFERENCES

1. Gelashvili, R., Spiegelman, A., Xiang, Z., Danezis, G., Li, Z., Xia, Y., Zhou, R., & Malkhi, D. (2022). Block-STM: Scaling blockchain execution by turning ordering curse to a performance blessing. *arXiv preprint arXiv:2203.06871*.
<https://doi.org/10.48550/arXiv.2203.06871>
2. Harris, T. L. (2001). A pragmatic implementation of non-blocking linked-lists.
<https://www.cl.cam.ac.uk/research/srg/netos/papers/2001-caslists.pdf>